

Neural MCPF Allocation

Project Status Report — Implementation & Benchmarking Phase

Course: Multi-Agent Systems | June 2026

Executive Summary

Project Status

The Neural MCPF Allocation project has completed eleven successive experiments, demonstrating that a compact transformer network can replicate the goal-allocation decisions of an exact combinatorial solver (RobustMCPF / LKH-TSP) at **2–4× single-instance speedup** and **1–6% execution-cost overhead** across 28 problem configurations ($N \in \{2, 3, 4, 5\}$ agents, $M \in \{2, \dots, 8\}$ goals, 8×8 grids). The most recent phase scales the model to 1.2M parameters and trains it on a diverse grid distribution, achieving a mean execution-cost ratio of **1.020** with 0% infeasible allocations across 5,600 test instances.

Phase Two — Real Benchmark Maps (Exps 12–15)

The project then moved onto the four 32×32 RobustMCPF benchmark maps (open, scattered, maze, rooms) at paper scale ($N \in \{5, 10, 15\}$, $M \in \{10, 20, 30\}$). Key results: map-trained models reach **1.048** mean execution-cost ratio; the old small-scale model *generalizes across map geometry but not across N/M scale*; training on cheap random grids *matches* fixed-map training everywhere *except the structured maze*; and at extreme scale ($N \leq 50$, $M \leq 100$) the larger model wins with **5.9–9.4×** speedup — the regime where replacing the solver pays off most.

Key Achievements

Architecture selection (Exp 1). A systematic comparison of three architectures on the base task ($N=2$, $M=2$, 5×5 grid) established the Transformer as the dominant model. It outperforms MLP and DeepSets on every metric, and its advantage over MLP grows from 1.7 pt at $N=2$ to 6.7 pt at $N=5$ — indicating that the attention mechanism better captures the combinatorial structure of larger problems.

Critical feature discovery (Exp 5). Adding goal-to-goal BFS distances $G \in \mathbb{R}^{M \times M}$ to the input lifted full-assignment accuracy by **14–18 percentage points** across all agent counts. This single input change outweighed every architecture, capacity, and data-scale improvement combined by a factor of $> 10\times$.

Universal mixed-size model (Exp 7). A single 151k-parameter transformer trained jointly on 15 problem configurations achieved zero-shot transfer to unseen $N=5$ problems, matching or exceeding per-size specialists trained specifically on those sizes.

Full-pipeline integration (Exps 8–10). The NN allocation module was integrated into the complete MAPF pipeline (NN forward pass $\rightarrow$ tour ordering $\rightarrow$ CBS collision planning), enabling end-to-end execution-cost benchmarks against the full solver across all 28 configurations at 200 instances each. Zero instances were ultimately infeasible after the probability-ranked fallback chain.

Large-model scale-up (Exp 11). A 1.2M-parameter model trained on diverse grids (sizes 8–12, wall density 0.1–0.5, all 28 $N \times M$ pairs) reduced the mean execution-cost ratio to **1.020** on its training distribution, improved hard high- M cells by 2–3 points, and produced zero infeasible allocations.

Technical Progress & Methodology

Problem Formulation

The project treats goal allocation as a multi-agent Travelling Salesman Problem (mTSP): given N agents on a grid with M goals, assign every goal to exactly one agent so as to minimize the total tour cost. Unlike balanced assignment, agents are unconstrained in the number of goals they receive — the solver may assign both goals to one agent when doing so is cheaper than splitting them. Ground-truth labels are produced by RobustMCPF running LKH-TSP in BasicMAPF mode; the neural network imitates these labels and replaces the solver’s allocation step at inference time.

The network input $D \in \mathbb{R}^{N \times M}$ is the BFS shortest-path distance matrix from each agent to each goal, normalized to $[0, 1]$. A secondary input $G \in \mathbb{R}^{M \times M}$ captures goal-to-goal distances and is injected via a learned projection. The output $P \in \mathbb{R}^{N \times M}$ is a column-wise softmax: for each goal j , $\sum_i P_{ij} = 1$, giving the probability that agent i visits goal j .

Loss Function

Training minimizes a composite loss:

$$\mathcal{L} = \mathcal{L}_{CE} + \lambda \cdot \mathcal{L}_{MinSum}$$

where $\mathcal{L}_{CE}$ is per-goal cross-entropy over agents (solver imitation), $\mathcal{L}_{MinSum} = \frac{1}{M} \sum_{i,j} P_{ij} D_{ij}$ penalizes distant assignments, and $\lambda = 0.1$. The cross-entropy term dominates in practice; the MinSum regularizer prevents degenerate uniform predictions early in training.

Architecture Evolution

Three architectures were evaluated. **GoalAllocMLP** flattens the input matrix and applies two hidden layers. **GoalAllocDeepSets** applies a shared per-goal MLP. **GoalAllocTransformer** applies alternating row-column attention on D with per-size positional embeddings. A fourth variant, **GoalAllocTransformerUniversal**, removes positional embeddings entirely and injects G via scalar embedding plus sum-pooling, making the model size-agnostic and enabling mixed-size training over heterogeneous (N, M) batches without padding.

Data Pipeline

Datasets are generated by placing agents and goals uniformly on random obstacle grids and calling the solver to produce labels. A BFS reachability filter discards instances where any goal is unreachable. In Exp 11, a CBS node budget (50k expansions per call) additionally rejects instances where constraint-dense mazes would prevent solver termination — a condition encountered in 12 of 28 generation tasks at 50% wall density. The pipeline supports configurable grid dimensions, wall densities, and (N, M) pairs, with parallel worker processes to offset the 30–65 ms per-instance solver overhead.

Training Configuration

All experiments use Adam with ReduceLROnPlateau and 100 epochs unless noted. Exp 11 uses $\text{lr} = 5 \times 10^{-4}$, gradient clipping at 1.0, and 150 epochs on 840k training samples (30k per configuration across all 28 pairs). A **MixedSizeBatchSampler** ensures each batch is shape-homogeneous. Hardware: NVIDIA RTX 3090; training time ~ 25 min/seed for the small model, ~ 3 h/seed for the large model.

Blockers & Challenges

Seed leakage (resolved). Before 2026-06-11, the dataset builder seeded every split with the same base seed, causing validation and test instances to replay the training set’s RNG stream. The original PoC numbers (per-goal 0.855, full-assignment 0.720) were measured on

contaminated test sets and are void. The fix introduces per-split seeding; a regression test in `tests/test_build_dataset.py` guards against recurrence.

CBS non-termination on dense mazes (resolved). At wall densities up to 50%, CBS can enter an infinite constraint-escalation loop on instances where no collision-free plan exists. This halted 12 of 28 data-generation tasks in Exp 11 and one evaluation instance in Exp 10. A 50k-node expansion budget now terminates CBS early; budget-exceeded instances are rejected during generation or resolved via the probability-ranked fallback chain at inference.

LKH binary portability. The LKH TSP binary is not committed to the repository (platform-specific). Cluster jobs invoke `scripts/setup_robustmcpf.sh` on first run, adding a one-time build cost on any new cluster node.

Metric Caveat

The offline cost ratio in Exps 1–7 is a first-hop proxy ($\sum P_{ij}D_{ij}$), not the true mTSP tour cost. Values below 1.0 in that metric reflect the metric mismatch, not super-solver performance. Full-pipeline experiments (Exps 8–11) use real CBS execution cost and are directly interpretable: ratio 1.0 means the NN matches the exact solver.

Brute-force tour ordering. The NN pipeline enumerates all $k!$ visit orders for each agent’s assigned goals. At $k = 7$ or $k = 8$ this dominates pipeline latency (38 ms for n2m8, 41 ms for n5m5), masking the NN’s allocation speed advantage. A learned ordering step would eliminate this bottleneck.

Data & Metrics

Metric Definitions

The experiments report two families of metrics. *Allocation-quality* metrics (per-goal and full-assignment accuracy, computed in `evaluation/evaluate.py`) compare the NN’s predicted assignment against the solver’s ground-truth assignment, with no path planning (Exps 1–7). *Execution-cost* metrics (cost ratio, exact match, diff, speedup, infeasible/fallback, computed in `evaluation/full_pipeline_eval.py`) run each pipeline end-to-end through CBS path planning and compare the resulting plans (Exps 8–11).

Notation. For one instance the network outputs $P \in [0, 1]^{N \times M}$ — each column a distribution over the N agents, $\sum_i P_{ij} = 1$ — and the solver gives ground truth $Y \in \{0, 1\}^{N \times M}$, each column one-hot. Write the predicted and true agent for goal j as $\hat{a}_j = \arg \max_i P_{ij}$ and $a_j^* = \arg \max_i Y_{ij}$, and the decoded hard allocation as $\hat{Y}_{ij} = \mathbb{I}[i = \hat{a}_j]$. $D \in \mathbb{R}^{N \times M}$ is the agent→goal BFS distance matrix, and $\langle \cdot \rangle$ denotes the arithmetic mean over the S test instances of a configuration. $\mathbb{I}[\cdot]$ is the 0/1 indicator.

Allocation-quality metrics (Exps 1–7) compare the predicted and ground-truth allocations directly, with no path planning. *Per-goal accuracy* is the fraction of individual goals routed to the solver’s chosen agent: per instance it is $\frac{1}{M} \sum_{j=1}^M \mathbb{I}[\hat{a}_j = a_j^*]$, and the reported figure is the mean of this over all test instances, $\langle \cdot \rangle$. It is lenient — one wrong goal in an M -goal instance still scores $(M-1)/M$ — and because M is fixed within a configuration it equals the overall share of correctly assigned goals. *Full-assignment accuracy* is the strict counterpart: the per-instance score is the all-or-nothing indicator $\mathbb{I}[\hat{Y} = Y]$, equal to 1 only when every one of the M decoded columns matches the solver, so its mean over instances empirically tracks (per-goal accuracy) M . *Cost ratio (proxy)* compares only the first-hop BFS cost of the two allocations: per instance it is the ratio $(\sum_{ij} D_{ij} \hat{Y}_{ij}) / (\sum_{ij} D_{ij} Y_{ij})$ — NN cost over solver cost, set to 1 when the solver cost underflows below 10^{-9} — and these per-instance ratios are then averaged. Because it ignores tour chaining and collisions, values below 1.0 reflect the metric mismatch, not the NN beating the solver.

Execution-cost metrics (Exps 8–11) instead run both pipelines end-to-end and compare the resulting plans, where the cost c of a plan is the total collision-free path length in integer grid steps that CBS returns after resolving every conflict (the solver’s `Solution`[5]). *Cost ratio (execution)* is the per-instance ratio c_{nn}/c_{sol} averaged over instances: 1.0 means the NN matches the optimal solver and 1.05 is 5% costlier. *Exact match* is the fraction of instances whose two integer costs are exactly equal, $\langle \mathbb{I}[c_{nn} = c_{sol}] \rangle$; it runs higher than full-assignment accuracy because many distinct allocations are cost-equivalent ties (for example, two agents equidistant from a goal). *Diff (steps)* is the mean signed gap $\langle c_{nn} - c_{sol} \rangle$ in grid steps, zero on an exact match, and the same per-instance differences also produce the std and max quoted in the logs. *Speedup* is the ratio of *mean* single-instance wall-times, $\langle t_{sol} \rangle / \langle t_{nn} \rangle$ — a ratio of means, not the mean of per-instance ratios — where t_{nn} covers the forward pass, goal-visit ordering and CBS, and t_{sol} covers LKH-TSP and CBS on the same CPU node. Finally, two robustness counters describe the probability-ranked candidate chain. *Infeasible* is the share n_{inf}/n_{done} of instances in which *all* $K=3$ ranked candidate allocations admit no collision-free plan (these carry no cost and are dropped from every other metric), and *Fallback* is the share in which the arg-max ($k=1$) allocation failed but a lower-probability candidate — $k \geq 2$, enumerated best-first over the joint probability $\prod_j P_{a_j,j}$ — produced a valid plan.

Architecture Comparison (Exp 1)

Table 1: Exp 1 — architecture comparison: N=2, M=2, 5×5, 5k train, 100 epochs, 3 seeds.

Model	Per-goal Acc	Full-assign Acc	Cost ratio (proxy)
DeepSets	0.858 ± 0.000	0.727 ± 0.000	0.969 ± 0.000
MLP	0.875 ± 0.004	0.756 ± 0.005	0.976 ± 0.001
Transformer	0.887 ± 0.001	0.782 ± 0.002	0.991 ± 0.002

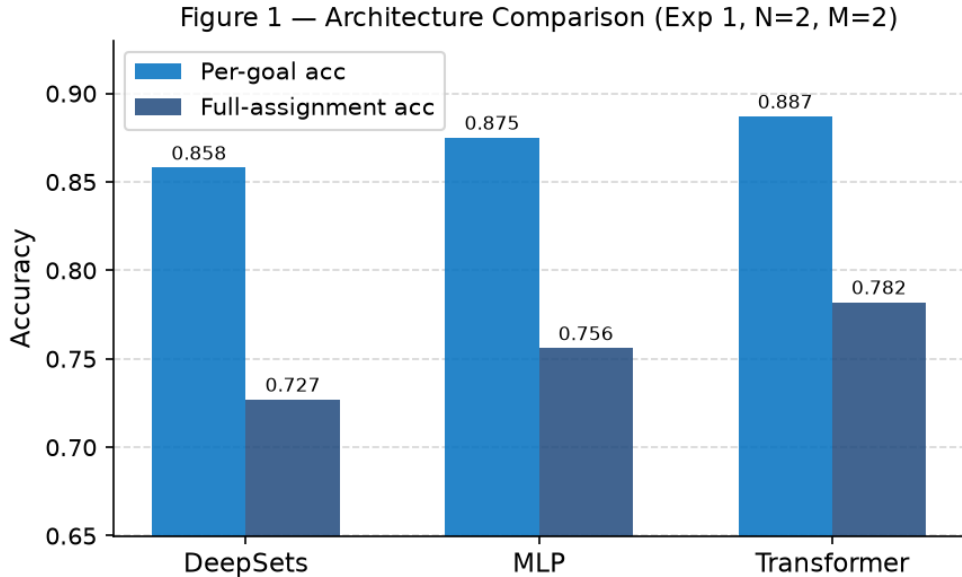


Figure 1: Architecture comparison (Exp 1). The Transformer achieves the highest accuracy on both metrics. The advantage over MLP widens from 1.7 pt at N=2 to 6.7 pt at N=5 (Table 2).

Table 2: Exp 2 — full-assignment accuracy (per-goal in parentheses): N scaling, 8×8, 10k train.

N	MLP	Transformer
2	0.772 ± 0.003 (0.882)	0.789 ± 0.006 (0.890)
3	0.549 ± 0.005 (0.812)	0.590 ± 0.004 (0.830)
4	0.382 ± 0.002 (0.776)	0.443 ± 0.008 (0.799)
5	0.270 ± 0.002 (0.758)	0.337 ± 0.004 (0.785)

N-Scaling (Exp 2)

Full-assignment accuracy decays roughly as (per-goal)^M for both models. The Transformer gap over MLP grows monotonically with N, suggesting attention better handles the cross-goal interdependencies that arise in larger problems.

Capacity and Data-Scale (Exps 3–4)

Table 3: Exp 3 — data-scale ablation: transformer h64/L3, N=3, 8×8.

Train size	Per-goal Acc	Full-assign Acc
1k	0.814 ± 0.002	0.553 ± 0.003
5k	0.824 ± 0.001	0.578 ± 0.001
10k	0.831 ± 0.003	0.599 ± 0.005
20k	0.835 ± 0.002	0.603 ± 0.003

Gains flatten by 20k samples (+0.4 pt from 10k to 20k). Similarly, widening the hidden dimension from 64 to 256 or deepening from L=3 to L=6 (Exp 4) adds at most +1.4 pt; h256/L6 diverges on some seeds at lr= 10⁻³. Together these results confirm that capacity and data alone cannot close the accuracy gap — the *G* feature is the critical missing piece.

Goal–Goal Distance Feature (Exp 5)

Table 4: Exp 5 — *G* ablation: transformer h64/L3, 8×8, 10k train, 3 seeds. Full-assignment accuracy.

N	Without <i>G</i>	With <i>G</i>	Gain
2	0.790 ± 0.002	0.932 ± 0.005	+14.2 pt
3	0.595 ± 0.010	0.775 ± 0.008	+18.0 pt
4	0.455 ± 0.006	0.632 ± 0.007	+17.7 pt
5	0.337 ± 0.004	0.478 ± 0.011	+14.1 pt

Key Finding: Goal–Goal Distances Dominate

The matrix *G* provides the *tour-structure signal* that *D* alone cannot express: which goals are adjacent and should be bundled to the same agent. A gain of +14–18 pt exceeds all architecture, capacity, and data-scale improvements combined by > 10×. This was the decisive design insight of the project.

Universal Mixed-Size Model (Exp 7)

The universal model outperforms per-size specialists on all configurations except the easiest (N=M=2, −1.9 pt), and achieves +3.5 pt on N=5 — a configuration it was *never trained on*.

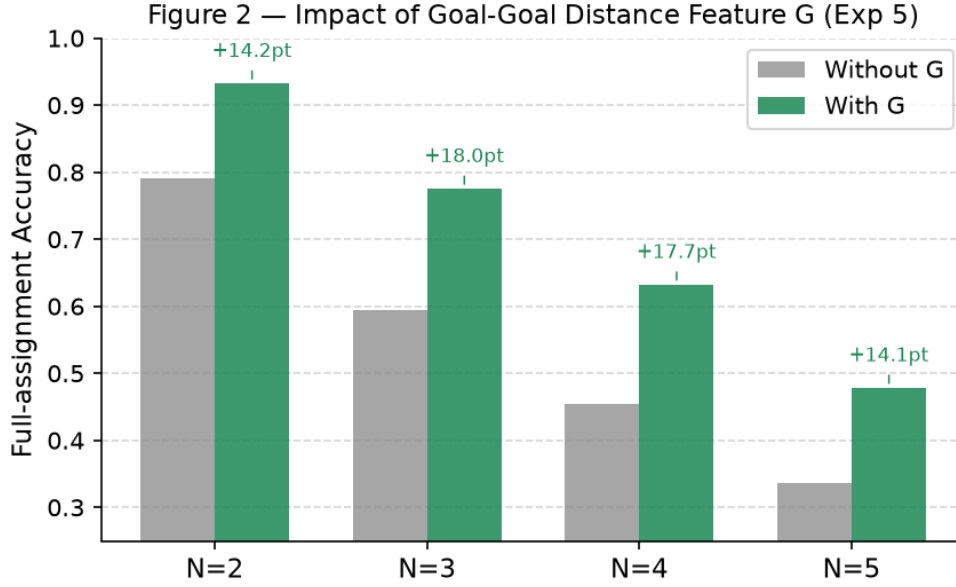


Figure 2: Adding goal-to-goal distances G improves full-assignment accuracy by 14–18 pt at every agent count. At $N=2$ with G , the mean cost ratio reaches 1.0000 — near-perfect solver agreement.

Table 5: Universal model (151k params, $N \in \{2, 3, 4\} \times M \in \{2..6\}$) vs. per-size specialists. Full-assignment accuracy, 3 seeds. $N=5$ is zero-shot.

Config	Per-size Specialist	Universal Model	Δ
$N=M=2$	0.932 ± 0.005	0.913 ± 0.002	−1.9 pt
$N=M=3$	0.775 ± 0.008	0.810 ± 0.002	+3.5 pt
$N=M=4$	0.632 ± 0.007	0.656 ± 0.007	+2.4 pt
$N=M=5$ (<i>zero-shot</i>)	0.478 ± 0.011	0.513 ± 0.003	+3.5 pt

Cross-configuration sharing acts as regularization, preventing the model from over-specializing to any single (N, M) pair and enabling genuine transfer to unseen sizes.

Full-Pipeline Evaluation (Exps 9–10)

Table 6: Exp 9 — end-to-end MAPF execution cost: NN pipeline vs. solver, 200 instances/config, single-instance timing. Execution cost = CBS collision-free path length (true MAPF objective).

Config	Cost tio	Ra- tio	Exact Match	Diff (steps)	NN ms	Solver ms	Speedup
n2m2	1.010		96.5%	0.08	9.6	16.2	1.7×
n3m3	1.007		97.0%	0.07	8.0	21.7	2.7×
n4m4	1.029		88.0%	0.27	9.2	28.3	3.1×
n2m4	1.020		88.5%	0.26	6.8	15.1	2.2×
n4m6	1.050		72.5%	0.69	10.0	36.3	3.6×
n2m6	1.063		64.0%	1.09	9.2	22.5	2.4×

Two structural insights emerge. First, exact-cost match (64–97%) is far higher than exact-assignment match because many differing allocations achieve the same CBS execution cost — they are cost-equivalent ties. Second, single-instance speedup (1.7–3.6×) is far below batched allocation-only speedup ($\sim 10^3\times$ in Exp 8): CBS path planning dominates both pipelines and only the LKH call is saved. Speedup trends upward with M as the solver’s TSP component grows while the NN forward pass remains constant-time.

Large Model on Diverse Distribution (Exp 11)

Table 7: Exp 11 offline per-goal accuracy on the diverse test set (h128/L6, 3 seeds, $\text{std}\approx 0.000$). Full-assignment accuracy in parentheses.

N\M	2	3	4	5	6	7	8
2	0.960 (.94)	0.949 (.90)	0.928 (.84)	0.911 (.76)	0.892 (.69)	0.876 (.63)	0.852 (.55)
3	0.930 (.89)	0.926 (.85)	0.915 (.79)	0.897 (.71)	0.881 (.65)	0.864 (.57)	0.850 (.51)
4	0.923 (.87)	0.919 (.82)	0.899 (.73)	0.886 (.67)	0.875 (.60)	0.854 (.51)	0.841 (.47)
5	0.921 (.86)	0.910 (.79)	0.903 (.73)	0.887 (.65)	0.877 (.58)	0.860 (.51)	0.845 (.44)

Accuracy degrades gracefully with M and is essentially flat across N . The $N=5$ row, which was zero-shot for the small model, is now fully in-distribution and matches $N=2$ at every M .

Table 8: Exp 11 full-pipeline summary: large model vs. small universal model (Exp 10), mean over 28 configs, 200 instances each.

Metric	Small (Exp 10, 8×8)	Large (8×8)	Large (diverse)
Mean cost ratio	~ 1.040	1.029	1.020
Exact-cost match	$\sim 80\%$	82.9%	85.0%
Infeasible alloc	$<0.5\%$	0%	0%
Fallbacks needed	$<0.5\%$	0%	0%
Mean speedup	$\sim 2.8\times$	2.4×	2.1×

The improvement of the large model concentrates on the hard, high- M configurations where the small model struggled most; easy small- M configs move at most ± 1 pt. On its own distribution the model achieves cost ratio 1.020, better than the off-distribution 8×8 evaluation (1.029), confirming good generalization across grid geometries.

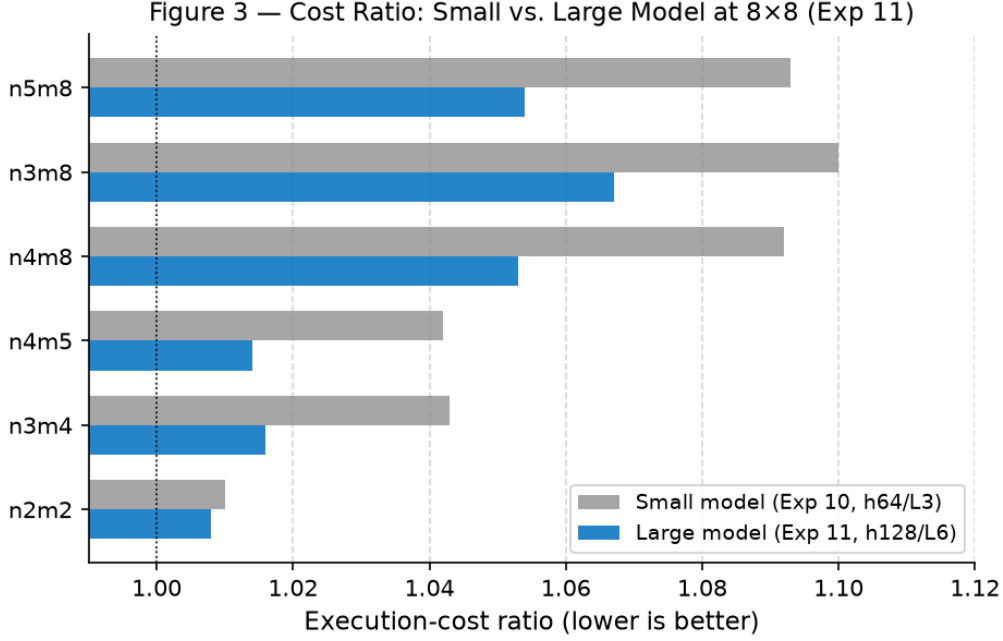


Figure 3: Cost ratio for selected configurations: small (Exp 10) vs. large (Exp 11) model. Improvements concentrate on hard high- M cells (e.g. n4m8: 1.092→1.053; n3m4: 1.043→1.016). A dotted line marks ratio=1.0 (exact-solver agreement).

Per-Configuration Breakdown: $N \times M$ Heatmaps (Exp 11, diverse)

Figures 4 and 5 render the two headline metrics across the full 28-configuration grid on the model’s own diverse distribution. Both make the central finding immediately visual: **the gradient runs horizontally, not vertically**. Difficulty is driven almost entirely by the number of goals M (each added goal is another per-goal decision that must be correct), while the agent count N leaves the metrics essentially flat — the same column colour holds from $N=2$ to $N=5$. The complete per-config tables backing these heatmaps appear in Appendix 6.2.

Training Convergence (Exp 11, large model)

Figure 6 traces the loss and per-goal accuracy of the h128/L6 model over its 150 training epochs, averaged across the three seeds (the ± 1 std bands are barely visible, so the runs are highly reproducible). The two panels expose a clear **overfitting** regime that shapes how the model is selected. Training loss falls monotonically from 0.39 to 0.09 and training accuracy climbs to 0.969, but the validation loss bottoms out at ≈ 0.25 around epoch 22 and then *rises* steadily to ≈ 0.85 , while validation accuracy peaks near 0.90 and settles at 0.884. The MinSum component stays essentially flat (≈ 0.257) throughout — the movement is almost entirely in the cross-entropy term, i.e. the model is progressively memorizing the solver’s exact label on the training instances.

Because `train.py` checkpoints the weights at the *minimum validation loss* rather than the final epoch, the deployed model is effectively **early-stopped** at its epoch-22 optimum (best val loss 0.2514/0.2528/0.2537 across seeds); all evaluation numbers in this report come from that checkpoint, not the over-trained final state. The 150-epoch schedule is therefore longer than strictly necessary — accuracy and loss have both plateaued by epoch ~ 80 — and a tighter early-stopping patience or stronger regularization is a natural lever for future runs (see Next Steps).

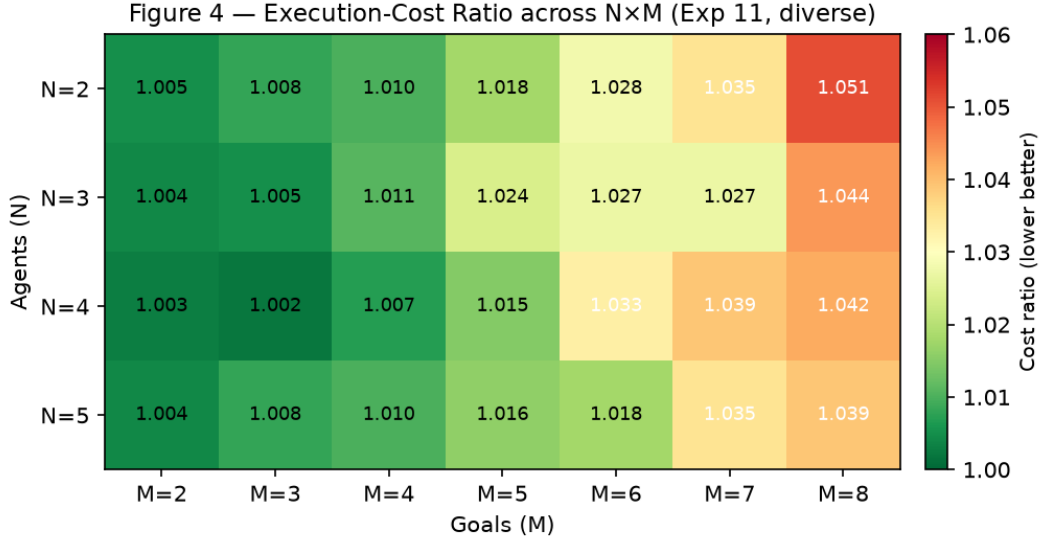


Figure 4: Execution-cost ratio across the $N \times M$ grid (Exp 11, diverse distribution, 200 instances/-config). Green = near-optimal (ratio $\rightarrow 1.0$), red = costlier. The worst cell (n2m8, 1.051) is still within $\sim 5\%$ of the exact solver.

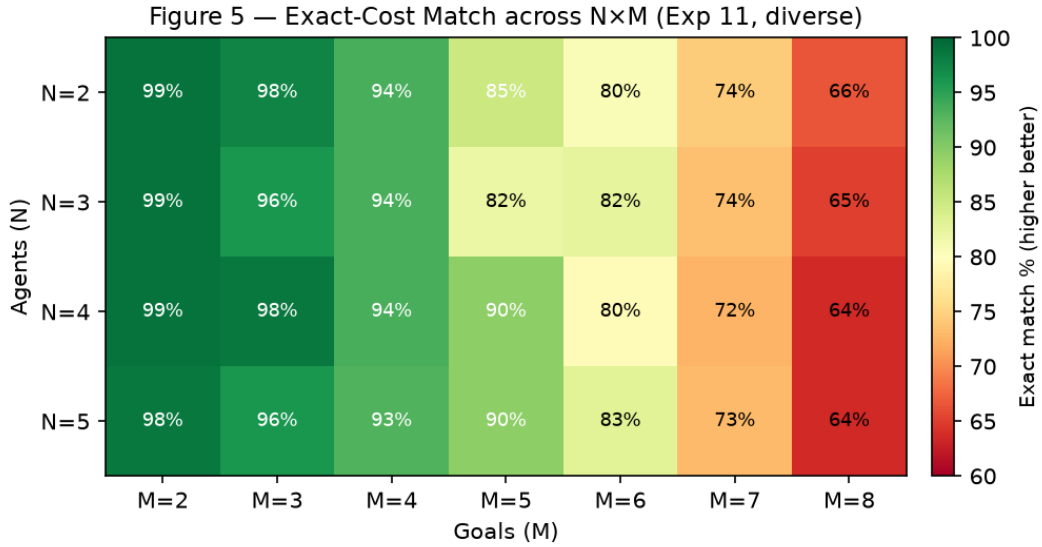


Figure 5: Exact-cost-match fraction across the $N \times M$ grid (Exp 11, diverse distribution). Green = high agreement with the solver’s execution cost. The 60–99% range tracks M almost perfectly and is near-constant in N .

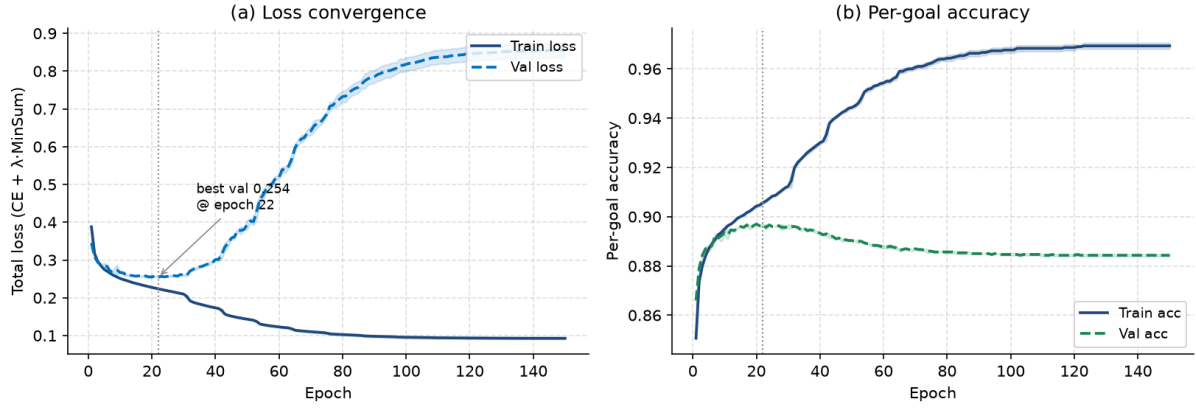
Figure 6 — Training Convergence, large model h128/L6 (mean of 3 seeds, ± 1 std)

Figure 6: Training convergence of the large model (h128/L6, mean of 3 seeds, ± 1 std). **(a)** Total loss: train (solid) keeps falling while validation (dashed) turns upward after its epoch-22 minimum — the checkpoint is taken at that minimum (dotted line). **(b)** Per-goal accuracy: the train/val gap widens after the same point. Both signatures are textbook overfitting, controlled here by best-val-loss checkpointing.

Full Per-Config Pipeline Tables (Exps 10–11)

The two full-pipeline sweeps are reproduced here in ascending experiment order: first the Exp 10 baseline (small universal model, with zero-shot extrapolation cells), then the Exp 11 final model whose N×M heatmaps appear in Figures 4–5.

Exp 10 — small universal model (h64/L3), 8×8

Table 9: Complete Exp 10: universal model (h64/L3), 28 configs, 200 instances/config, 8×8/10% obstacles. ZS = zero-shot ($N=5$ or $M \geq 7$).

Config	ZS	Cost Ratio	Exact Match	Diff (steps)	NN ms	Speedup
n2m2		1.010	96.5%	0.07	11.3	3.0×
n2m3		1.006	98.0%	0.08	12.2	2.7×
n2m4		1.020	88.5%	0.26	12.7	2.5×
n2m5		1.044	74.5%	0.69	12.2	3.1×
n2m6		1.063	64.0%	1.09	17.0	2.3×
n2m7	✓	1.090	54.5%	1.61	18.3	2.1×
n2m8	✓	1.078	51.0%	1.55	38.7	1.1×
n3m2		1.009	98.0%	0.07	13.3	2.6×
n3m3		1.007	97.0%	0.07	12.1	2.6×
n3m4		1.043	83.0%	0.47	11.7	3.4×
n3m5		1.046	76.5%	0.60	12.3	3.2×
n3m6		1.070	66.5%	1.03	12.9	3.2×
n3m7	✓	1.068	58.0%	1.12	16.4	2.7×
n3m8	✓	1.100	43.5%	1.82	20.4	2.5×
n4m2		1.000	100.0%	0.00	10.9	3.1×
n4m3		1.010	96.0%	0.09	11.7	3.2×
n4m4		1.029	88.0%	0.27	12.1	3.3×
n4m5		1.042	76.0%	0.54	12.8	3.5×

Table 9 continued

Config	ZS	Cost Ratio	Exact Match	Diff (steps)	NN ms	Speedup
n4m6		1.050	72.5%	0.69	13.3	3.5×
n4m7	✓	1.074	55.5%	1.09	19.8	2.5×
n4m8	✓	1.092	40.0%	1.52	19.6	2.7×
n5m2	✓	1.002	99.5%	0.01	34.8	1.1×
n5m3	✓	1.009	96.5%	0.07	30.1	1.3×
n5m4	✓	1.020	91.5%	0.19	13.3	3.3×
n5m5	✓	1.041	76.5%	0.47	41.5	1.2×
n5m6	✓	1.053	73.0%	0.67	14.3	3.9×
n5m7	✓	1.070	52.5%	0.98	15.5	4.1×
n5m8	✓	1.093	38.5%	1.49	17.4	3.8×

Exp 11 — large model (h128/L6), diverse distribution

Table 10: Complete Exp 11: large model (h128/L6), 28 configs, 200 instances/config, on its own diverse distribution (grids 8–12, walls 0.1–0.5). All configs in-distribution (no extrapolation column). These are the values plotted in Figures 4–5.

Config	Cost Ratio	Exact Match	Diff (steps)	Speedup
n2m2	1.005	99.0%	0.04	1.9×
n2m3	1.008	97.5%	0.12	1.9×
n2m4	1.010	93.5%	0.17	1.7×
n2m5	1.018	85.0%	0.42	1.8×
n2m6	1.028	80.5%	0.66	2.1×
n2m7	1.035	74.5%	0.81	2.1×
n2m8	1.051	66.5%	1.39	1.3×
n3m2	1.004	99.0%	0.04	2.2×
n3m3	1.005	96.0%	0.06	1.9×
n3m4	1.011	93.5%	0.17	2.1×
n3m5	1.024	82.0%	0.49	2.0×
n3m6	1.027	82.5%	0.55	2.3×
n3m7	1.027	73.5%	0.55	2.1×
n3m8	1.044	65.0%	1.08	2.1×
n4m2	1.003	99.0%	0.02	2.6×
n4m3	1.002	98.5%	0.02	2.2×
n4m4	1.007	93.5%	0.09	2.3×
n4m5	1.015	89.5%	0.25	0.6×
n4m6	1.033	79.5%	0.63	2.3×
n4m7	1.039	72.5%	0.81	2.4×
n4m8	1.042	63.5%	0.97	2.7×
n5m2	1.004	98.5%	0.02	2.3×
n5m3	1.008	96.0%	0.08	2.4×
n5m4	1.010	93.0%	0.15	2.5×
n5m5	1.016	89.5%	0.22	1.1×
n5m6	1.018	83.0%	0.34	2.3×
n5m7	1.035	73.0%	0.67	2.8×
n5m8	1.039	63.5%	0.83	2.7×

Real Benchmark Maps and Training-Data Study (Exps 12–15)

The second phase replaces procedural grids with the four 32×32 RobustMCPF benchmark maps and scales to $N \in \{5, 10, 15\}$, $M \in \{10, 20, 30\}$. Figure 7 shows the four maps, spanning open geometry to highly structured corridors.

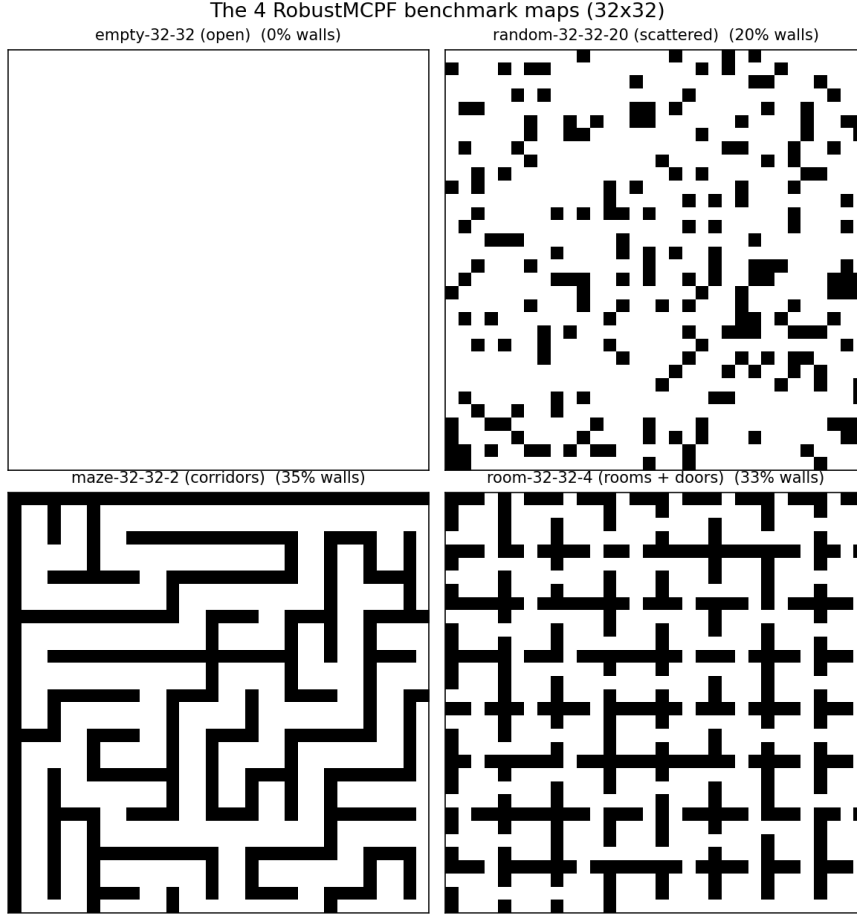


Figure 7: The four 32×32 benchmark maps. Only maze and room carry *structured* (corridor/room) geometry — the distinction that drives the Exp 14 result.

Map-Trained Models: h128 vs h256 (Exp 12)

Two universal models (the Exp-11 architecture) were trained directly on the four maps at the new scale. In-distribution the smaller **h128** model wins (mean execution-cost ratio **1.048** vs 1.052 for h256) — the extra capacity overfits rather than helping. As throughout, difficulty tracks M , not N .

Does the Old Model Transfer? (Exps 13.a, 13.b)

Exp 13.a evaluates the Exp-11 model (*trained only on small random grids*) zero-shot on the real maps at small N/M : it transfers cleanly — mean ratio **1.019**, and structured maps are *easier* than open ones because walls remove allocation ties. Exp 13.b pushes the same model to the large- N/M regime, where it *fails to extrapolate*: mean ratio **1.246** versus the map-trained B1’s 1.048. The signature is a scissors (Figure 8): at $M=30$, adding agents *hurts* the old model (trained on $N \leq 5$) but *helps* the in-range B1.

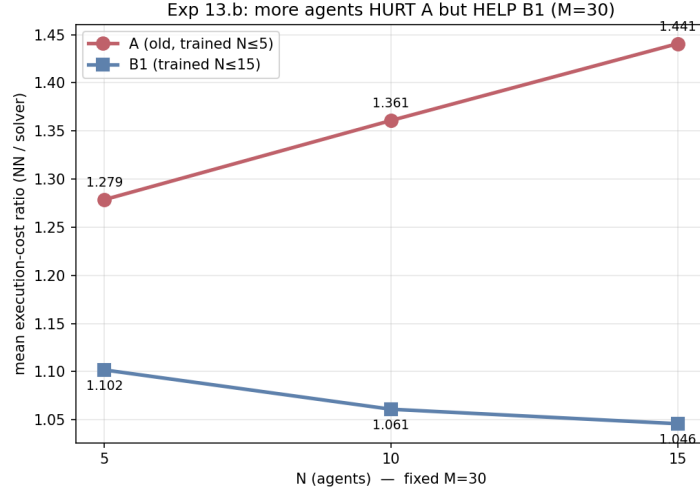


Figure 8: Exp 13.b: at $M=30$, more agents hurt the old model A (trained $N \leq 5$) but help the map-trained B1 (trained $N \leq 15$). Generalization fails across *scale*, not geometry.

Random-Diverse vs Fixed-Map Training (Exp 14)

A controlled study: models **C** copy the **B** recipe exactly, swapping only the training data — random 32×32 grids with 0–50% wall density instead of the four fixed maps. Fixed-map training wins overall (h128 1.048 vs 1.075), but **the entire gap is the maze** (Figure 9); on open/scattered/room maps the two tie. Random Bernoulli walls resemble those maps but never reproduce the maze’s corridor structure.

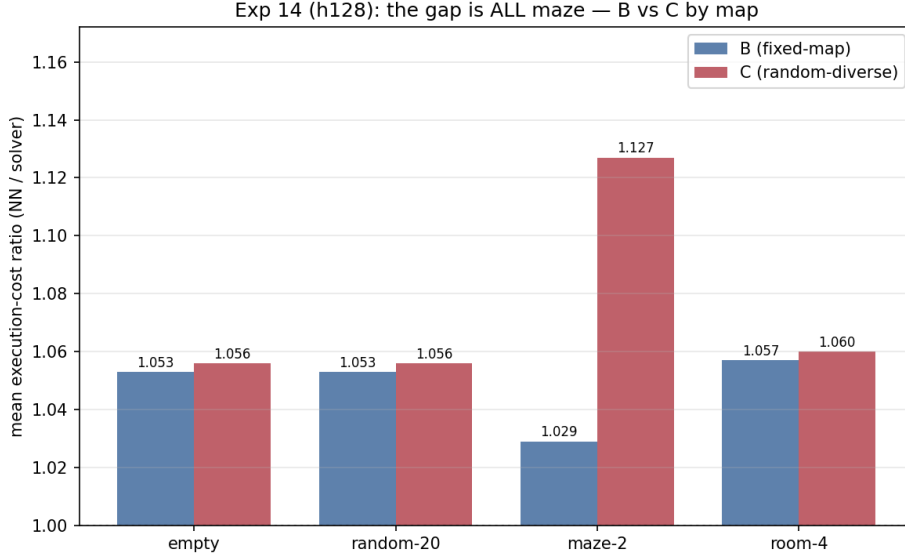


Figure 9: Exp 14 (h128): random-diverse training (C) ties fixed-map training (B) on every map *except* the maze, where it never saw corridor structure.

Zero-Shot XL Extrapolation (Exp 15)

The Exp-14 winner (the fixed-map family) is pushed far beyond its training range to paper scale, $N \in \{20, 35, 50\} \times M \in \{50, 75, 100\}$ (toward the paper’s $N \leq 50 / M \leq 100$), with no retraining. Here the in-distribution verdict *flips*: the larger **h256** model extrapolates better (mean cost ratio **1.208** vs 1.244 for h128) and roughly halves the worst-case gap, while single-instance speedup climbs to **5.9–9.4×** as the solver’s TSP/CBS workload grows. The capacity that merely overfit

in range becomes the better generalizer at scale — and this is the regime where replacing the exact solver yields the largest speed win.

Phase-Two Conclusion

Training data matters for *structured* geometry only. The old model generalizes across map shape but not problem scale; random-grid training is a free substitute for fixed-map training on unstructured maps but loses on the maze. At extreme scale (Exp 15) the larger model extrapolates best and runs **5.9–9.4×** faster than the solver — the operating point where replacing the exact solver is most worthwhile.

Summary: The Model and Where It Helps

How it works. Each instance is encoded as numeric matrices — the agent→goal BFS distances $D \in \mathbb{R}^{N \times M}$ and goal→goal distances $G \in \mathbb{R}^{M \times M}$. The universal transformer processes them with alternating row/column attention (no positional embeddings, so it accepts *any* N, M) and emits a column-wise softmax $P \in \mathbb{R}^{N \times M}$ — for each goal, a distribution over agents. An arg-max decode yields the allocation, which a classical goal-ordering and CBS step turns into collision-free paths. In short: *problem* $\rightarrow (D, G)$ *vectors* $\rightarrow$ *attention* $\rightarrow$ *per-goal probabilities* $\rightarrow$ *allocation* $\rightarrow$ *paths*.

Small vs. large model. Two sizes were studied: **h128/L6** (1.2M params) and **h256/L8** (6.3M). In-distribution the small model is both more accurate and faster — the large one overfits (Exp 12). Far out-of-distribution the ranking reverses: the large model is the better extrapolator (Exp 15). Practically, prefer h128 when deployment matches the training range and h256 when heavy extrapolation beyond it is expected.

When to use the NN instead of the exact solver. The network replaces only the combinatorial *allocation* step (LKH-TSP); collision planning (CBS) is shared by both pipelines. Its advantage therefore appears where allocation/TSP cost dominates and where many instances must be solved: (i) *large* N, M , where LKH grows expensive and the NN gives **5.9–9.4×** speedup at $\sim 20\%$ cost overhead (Exp 15); (ii) *batched, high-throughput* use, where allocation-only inference is orders of magnitude faster; and (iii) *unstructured geometry*, where even cheap random-grid training transfers (Exp 14). Conversely, on small instances the exact solver is already millisecond-fast with few collisions, so the NN offers no speed benefit (Exp 13.a); and on *structured* maps it must be trained on that geometry to stay competitive (the maze lesson, Exp 14). The NN is thus best understood as a fast, *near-optimal* drop-in for the allocation oracle ($\sim 1\text{--}6\%$ over optimal in range, $\sim 20\%$ at extreme scale) for use when speed and scale matter more than exactness.

Next Steps

Tour-aware loss. The solver’s true mTSP cost is computed during data generation and currently discarded. Storing it per sample is a one-line change to `build_dataset.py`. A tour-chain regularizer $\sum_{j,k} (\sum_i P_{ij} P_{ik}) G_{jk}$ would directly penalize splitting adjacent goals across agents, targeting the M-difficulty axis.

Learned goal-visit ordering. Replacing the $k!$ brute-force enumeration with a learned ordering head would eliminate the latency spike at high k and extend the NN’s speed advantage to arbitrary M without the $k = 7/8$ outlier problem.

Curb the overfitting. The convergence curves (Figure 6) show validation loss diverging after epoch ~ 22 while training loss keeps falling. Best-val-loss checkpointing already captures the right model, but explicit early stopping (e.g. patience ~ 20), weight decay, or dropout would shorten the 150-epoch schedule and likely lift the validation plateau itself rather than just stopping at it.

Collision-aware training. CBS execution cost is not differentiable, but approximate conflict penalties or policy-gradient signals could move training toward optimizing the true MAPF objective rather than the allocation-imitation proxy.

Report figures from raw data. The per-instance CSVs from Exp 11 on the cluster (5,600 rows total: `results/fullpipe_large/` and `results/fullpipe_large_diverse/`) support standard-deviation and percentile analyses without re-running any experiments. Generating final report-quality figures from this raw data is the immediate documentation task.